NBA Accredited & ISO 9001:2015 Certified
VIDYA VIKAS PRATISHTHAN, POLYTECHNIC SOLAPUR

Department of Computer Science & Engineering

A

SYNOPSIS REPORT

ON

"Project Orion: Multimodal Desktop & OS Automation Engine via WhatsApp Remote Gateway"

Has been
Submitted in partial fulfilment
of the Synopsis Report

By

Name	Enrollment No.
Shubham Dinesh Vernekar	24212130227

UNDER THE GUIDANCE OF

Prof. Hulsure A. G.

NBA Accredited & ISO 9001:2015 Certified

VIDYA VIKAS PRATISHTHAN POLYTECHNIC, SOLAPUR

CERTIFICATE

This is to certify that the Synopsis Report entitled

“Project Orion: Multimodal Desktop & OS Automation Engine via WhatsApp Remote Gateway”

Has been completed by

Name	Enrollment No.
Shubham Dinesh Vernekar	24212130227

of

T.Y. (Computer Science & Engineering)
Institute Code - 0854

Has been submitted in partial fulfilment of Synopsis Report as per the curriculum laid by M.S.B.T.E. Mumbai, during the academic year **2026-2027**

Prof. Hulsure A. G.
(GUIDE)

Prof. Buddhe M. V.
(H.O.D.)

NBA Accredited & ISO 9001:2015 Certified

VIDYA VIKAS PRATISHTHAN, POLYTECHNIC SOLAPUR

Department of Computer Science & Engineering

Academic year: - 2026-2027

CERTIFICATE

This is certifying that **Mr. Shubham Dinesh Vernekar** of **Third Year (CSE)** has successfully completed the project “Project Orion: Multimodal Desktop & OS Automation Engine via WhatsApp Remote Gateway” during the academic year 2026-27 in partial fulfilment of the requirements for Computer Science and Engineering.

Enrollment No: 24212130227

Name of Faculty	**Head of the Department**
Prof. Hulsure A. G.	Prof. Buddhe M. V.

Date: / /

ACKNOWLEDGEMENT

A synopsis report is a coordinated academic effort that becomes possible through guidance, encouragement and access to appropriate learning resources. I express my sincere gratitude to the Head of the Department, Prof. Buddhe M. V., and my project guide, Prof. Hulsure A. G., for their valuable guidance in selecting the topic, assessing its feasibility, understanding the problem and organizing this report.

I am thankful to the Principal, Dr. Kulkarni S. S., and all teaching and laboratory staff of V. V. P. Polytechnic, Solapur, for providing the academic environment and the software laboratory and computational facilities required for completing this work.

I also acknowledge the contribution of the researchers and publishers whose international research papers on conversational control, multimodal language models, tool-using agents, Windows automation and LLM application security helped in preparing the literature review.

Finally, I express my gratitude to my family and friends for their encouragement and support throughout the preparation of this report.

Date: / /

Place: Solapur

Shubham Dinesh Vernekar
Enrollment No.: 24212130227

Principal
Dr. Kulkarni S. S.

HOD
Prof. Buddhe M. V.

ABSTRACT

Remote administration of computers is moving toward conversational interfaces. Conventional remote-desktop tools stream continuous video, need a stable high-bandwidth connection and are awkward on a phone screen. The proposed “Project Orion” is an operating-system automation engine and secure remote bridge that a verified owner operates entirely through WhatsApp, using natural-language text, voice notes and short command tags. It turns a private WhatsApp chat into a command center for the owner's own Windows PC.

Orion is built on the Baileys WhatsApp multi-device library and Google Gemini multimodal models (Gemini 2.5 Flash / Pro) and runs natively on the host workstation. It allows the owner to inspect live hardware metrics (CPU load, RAM utilisation, drive capacity and uptime), receive desktop screenshots, organise folders, search and read files, launch applications, clear temporary files and the Recycle Bin, and run PowerShell and Command Prompt commands, all from a smartphone.

The methodology separates understanding from execution: an Authority Guard first verifies the sender, a regex fast-path or Gemini then classifies the request and extracts parameters, and a fixed set of native Windows subsystems (WMI, GDI+, filesystem engine, PowerShell runner) carries out the task locally. Orion has no subscription cost, opens no inbound port and exchanges only text and single images, so it needs very little bandwidth. This synopsis presents the architecture, technologies, algorithm, resources, advantages, limitations and future scope of the system.

Keywords: Artificial Intelligence, Gemini, WhatsApp Bot, Baileys, Desktop Automation, Remote Administration, WMI, PowerShell, Node.js, Authority Guard.

INDEX

Sr. No.	Contents	Page No.
1	Cover Page	1
2	Title Page	2
3	Certificate by the Guide	3
4	Acknowledgement	4
5	Abstract	5
6	Index	6
7	List of Figures	7
8	Introduction	8
9	Literature Review	9
10	System and Technology Overview	10
11	Core Modules and AI Model	11
12	Proposed Methodology and Algorithm	12
13	Command Processing Algorithm and Security	14
14	Advantages and Disadvantages	15
15	Conclusion	16
16	Project Initiation	17
17	Stakeholders and Resources	18
18	References	19

LIST OF FIGURES

Figure No.	Description	Page No.
Fig. 1	Proposed Project Orion System Architecture	10
Fig. 2	Orion Event-Processing Flowchart	13
Fig. 3	Intent Routing: Regex Fast-Path and Gemini	14

The figures are conceptual representations prepared for this synopsis report. They describe the proposed architecture and algorithm of Project Orion.

1. INTRODUCTION

1.1 Background

Smartphones have become the main digital interaction device, yet personal computers and workstations remain where heavy software, code bases and data live. When a user is away from the desk, simple tasks such as checking a background render, retrieving a forgotten file, taking a screenshot of an ongoing task or locking the workstation usually demand remote-desktop applications or SSH terminals, both of which are cumbersome on a phone.

The human-machine interface has progressed from physical switches to graphical user interfaces, touchscreens and, now, conversational natural-language interfaces. Project Orion brings the lightweight, end-to-end encrypted messaging channel of WhatsApp (through the Baileys multi-device library) together with Google Gemini's multimodal reasoning models, so that the user can talk to the operating system in plain text, voice notes and structured command tags.

1.2 Need of the System

- To remove the dependency on bandwidth-heavy remote-desktop software for everyday administrative tasks.
- To avoid recurring per-device subscription costs of commercial remote monitoring and management platforms.
- To replace precise tapping and zooming on a small screen with simple conversational commands.
- To keep command execution on the owner's own machine instead of relaying sessions through third-party servers.
- To understand voice notes and the intent of a request, instead of requiring every command to be typed manually.

1.3 Synopsis Purpose

The purpose of this synopsis is to study how a messaging gateway, a multimodal language model and native Windows interfaces can be combined to design a secure, low-cost, conversational OS automation engine. The report emphasizes the architecture, data flow, software platform, resources, security design, advantages, limitations and future scope.

2. LITERATURE REVIEW

Recent international research shows growing interest in controlling real systems through chat interfaces and language models, together with increasing concern for the security of such designs. The selected papers below were used as the main literature base for this synopsis.

1. Aina, Okegbile, Makanju and Oluwaranti (2022) – IGI Global
The authors present a chatbot-controlled home appliance system in which a Node.js server linked to Facebook Messenger classifies each message with Rasa NLU and sends the action over a WebSocket to a Raspberry Pi. It is the closest published architecture to Orion (messaging platform, intent classifier, local executor). Orion differs by controlling a full Windows PC and by using a multimodal language model.

2. Gemini Team, Google (2023) – arXiv:2312.11805
This technical report describes the Gemini family of models that accept text, images, audio and video and are strong at multilingual understanding. This multimodality allows Orion to handle voice notes and multilingual requests with a single model.

3. Schick et al. (2023) – NeurIPS (Toolformer)
The study shows that a language model can learn when and how to call external tools through APIs. Orion follows the same division of labour: it exposes a fixed set of tools (telemetry, screenshot, filesystem, shell) and relies on the model only to choose a tool and fill in its parameters.

4. Greshake et al. (2023) – arXiv:2302.12173
The paper shows that LLM-integrated applications blur the line between data and instructions and can be exploited through indirect prompt injection. Because Orion lets a language model influence actions on a real machine, it accepts input only from the verified owner and keeps shell access behind the Authority Guard.

5. Zhang et al. (2024) – UFO, Microsoft
UFO is an agent for Windows that observes application windows and carries out actions through the user interface. Orion differs in being reached through a remote chat gateway and in acting through native interfaces (WMI, PowerShell, filesystem) rather than by operating on-screen controls.

6. Xie et al. (2024) – NeurIPS (OSWorld)
OSWorld benchmarks multimodal agents on real computer tasks and reports that they still fall well short of human performance on open-ended screen-based tasks. This supports Orion's choice of a fixed set of reliable native tools instead of free-form screen control.

These studies collectively support the proposed concept: use a messaging channel as the interface, a multimodal model only to understand intent, and a fixed set of native tools with strict owner-only access to act on the machine.

3. SYSTEM AND TECHNOLOGY OVERVIEW

3.1 Proposed System

The proposed system accepts a text message, voice note or command tag from the owner's WhatsApp. The gateway layer receives it, the Authority Guard verifies the sender, and the intent router determines what is being asked. The matching native Windows subsystem then executes the task locally and the result is returned to WhatsApp as text or an image.

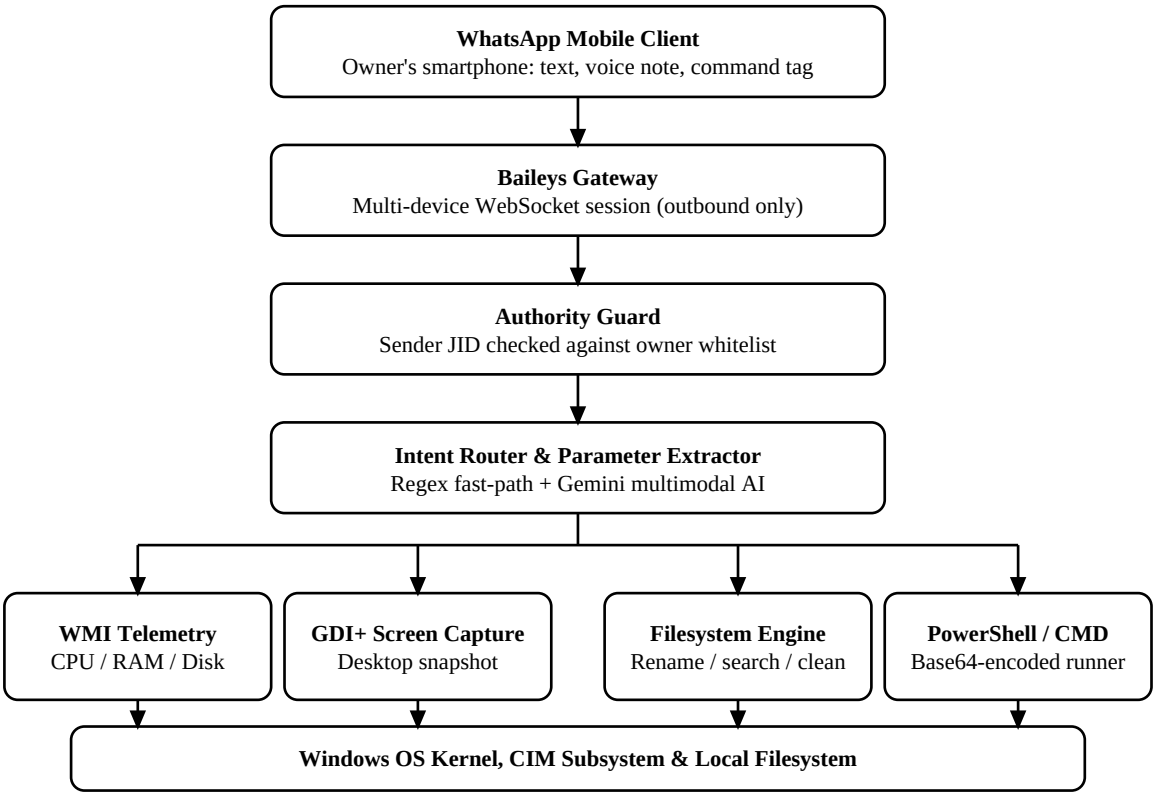


Fig. 1 – Proposed Project Orion System Architecture

3.2 Relevant Technologies

- Node.js (v18+) – event-driven core engine and package management through NPM.
- Baileys (@whiskeysockets/baileys) – WhatsApp multi-device WebSocket gateway.
- Google Gemini 2.5 Flash / Pro – multimodal reasoning, voice-note understanding and parameter extraction.
- Windows Management Instrumentation (WMI / CIM) – CPU, RAM, disk and uptime telemetry.
- GDI+ – desktop screen capture; user32.dll LockWorkStation – workstation lock.
- PowerShell 5.1+ / 7.x and Command Prompt – execution of administrative commands.
- Supporting libraries – docx, pdfkit, pino and dotenv.

4. CORE MODULES AND AI MODEL

4.1 Baileys Gateway

Baileys keeps a persistent multi-device WebSocket session with WhatsApp. Credentials stored in the auth_session folder allow the session to reconnect through a QR code or saved pairing keys. The connection is outbound only, so no listening port is exposed on the host.

4.2 Authority Guard

Every incoming event is checked: the sender's JID is compared with the owner whitelist. Messages from anyone else are dropped without a reply. This guard is the main access-control mechanism of the system.

4.3 Intent Router (Regex Fast-Path and Gemini)

Standard commands such as *pc status*, *take screenshot* and *lock pc* are recognised by regex heuristics within milliseconds without calling the cloud. Complex conversational requests and voice notes are sent to Gemini 2.5 Flash (with fallback models), which returns a chosen action and structured parameters.

4.4 Native Windows Subsystems

- **WMI telemetry:** live CPU load, RAM utilisation, drive capacity and uptime.
- **GDI+ capture:** single-frame desktop screenshots sent back as images.
- **Filesystem engine:** create, read, search, batch-rename and clean files and folders; Recycle Bin purge.
- **PowerShell / CMD runner:** commands are Base64 (UTF-16LE) encoded so that quotes and special characters cannot corrupt the command line.

4.5 Why a Hybrid Approach?

The language model understands what the user wants; the native subsystems actually do it. Combining both lets the system accept natural speech while keeping every action inside a small, predictable set of tools. Fast-path commands stay local and quick, and this separation also makes the design easier to explain, test and secure.

5. PROPOSED METHODOLOGY AND ALGORITHM

5.1 Data Inputs

- WhatsApp text messages from the owner.
- Voice notes in English, Hindi or regional languages, transcribed and understood by Gemini.
- Short command tags such as #ps (PowerShell) and #cmd (Command Prompt).
- Sender identity (JID) attached to every incoming event.
- Live host data read on demand: CPU load, RAM %, disk capacity, uptime and screen frame.

5.2 Message Processing

1. Receive the event through the Baileys WebSocket.
2. Verify the sender's JID against the owner whitelist; drop the event if it does not match.
3. Try the regex fast-path; if no pattern matches, send the text or audio to Gemini.
4. Extract the chosen action and its parameters in structured form.
5. Validate that the action belongs to the fixed tool set before running it.

5.3 Execution and Evaluation

The matching subsystem executes the task locally and the formatted result or media file is sent back to WhatsApp. Performance can be assessed against the project's target response times (telemetry under 2 seconds, screenshot under 3 seconds), the accuracy of intent classification for text and voice commands, and the correctness of file and shell operations.

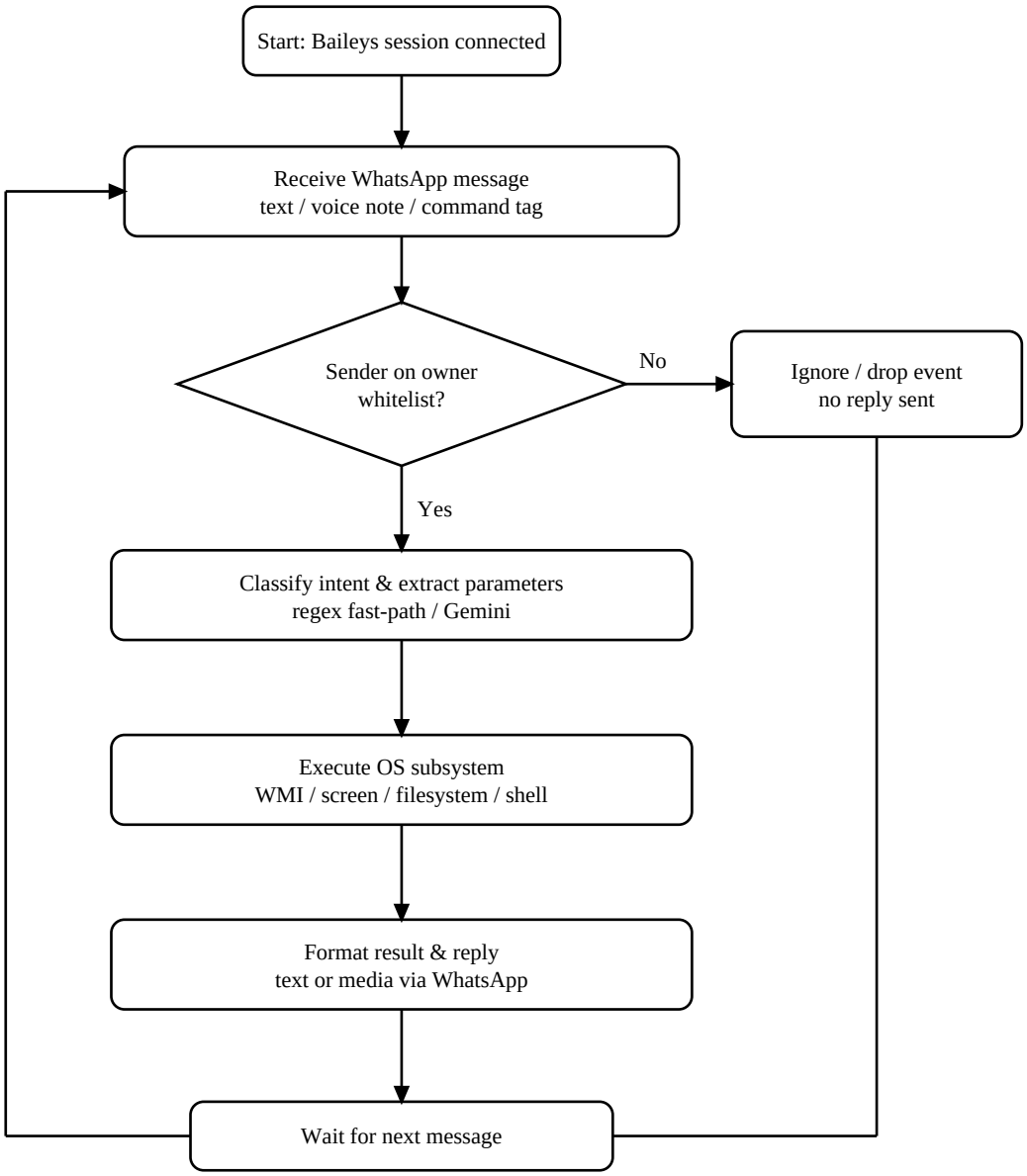


Fig. 2 – Orion Event-Processing Flowchart

6. COMMAND PROCESSING ALGORITHM AND SECURITY

6.1 Intent Routing

After the sender is verified, the router first tries the regex fast-path. Only requests that do not match are sent to Gemini, which selects an action and fills in its parameters. The model never runs commands itself; it only chooses among the tools Orion exposes.

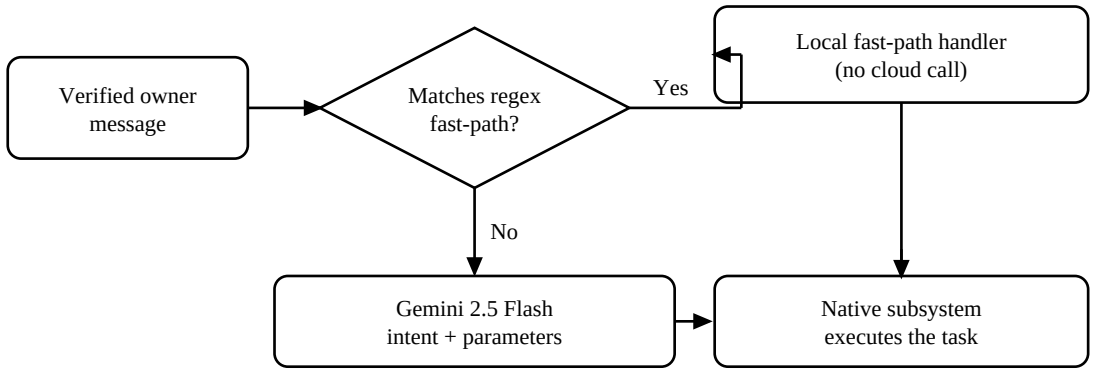


Fig. 3 – Intent Routing: Regex Fast-Path and Gemini

6.2 Algorithm Steps

- Connect the Baileys session and wait for a WhatsApp message.
- Receive text, voice note or command tag and read the sender JID.
- If the sender is not on the owner whitelist, drop the event with no reply.
- Classify the intent using the regex fast-path or Gemini and extract parameters.
- Execute the matching subsystem (WMI, screen capture, filesystem or shell) on the host.
- Format the result and send it back as text or media, then wait for the next message.

6.3 Security Considerations

- **Owner-only access:** messages from anyone else are dropped without a reply.
- **No inbound ports:** the gateway uses an outbound WebSocket connection only.
- **Local execution:** fast-path commands never leave the machine; only messages routed to Gemini do.
- **Encoded execution:** Base64 encoding protects the command line but does not limit what an authorised command can do, so access control rests on the Authority Guard.
- **Prompt-injection awareness:** the model only selects an action and parameters, and only the owner's messages are accepted as input.

7. ADVANTAGES AND DISADVANTAGES

7.1 Advantages

- **No subscription cost:** uses open-source libraries and the Gemini free tier.
- **Low bandwidth:** exchanges only text and single images instead of continuous video.
- **Local execution:** commands run on the owner's machine without opening any inbound port.
- **Conversational and multilingual:** understands text and voice notes in English, Hindi and regional languages.
- **Owner-only security model:** every message is checked by the Authority Guard before any action.
- **Modular design:** gateway, router and subsystems can be improved or extended independently.
- **Useful for personal productivity** and small-scale IT administration.

7.2 Disadvantages / Limitations

- Works only on Windows and does not provide continuous video streaming.
- Depends on an active internet connection, on WhatsApp and on the open-source Baileys library continuing to work with it.
- Messages routed to Gemini leave the machine, and the free tier has usage limits.
- Language-model interpretation is probabilistic, so a request may occasionally be misunderstood.
- Shell access gives broad authority; if the owner's phone or WhatsApp account is compromised, the PC is exposed.
- Access control is based on the sender's identity alone, with no second authentication factor in the current design.

7.3 Safety Principle

Orion should be presented as an owner-controlled administration tool. It must be used only on the owner's own computer, the owner must protect the paired phone and WhatsApp account, and destructive commands should be issued with care.

8. CONCLUSION

Project Orion proposes a lightweight alternative to graphical remote-desktop tools: a private WhatsApp chat through which the verified owner can monitor, manage and automate a Windows PC using natural language and voice notes. By combining the Baileys gateway, Google Gemini multimodal reasoning and native Windows subsystems, the system keeps execution local, needs very little bandwidth and carries no subscription cost.

The literature indicates that messaging platforms can serve as control channels, that language models can choose tools reliably when the tool set is fixed, and that LLM-integrated applications need strict input control. Orion applies these ideas through an Authority Guard, a fixed set of native tools and a hybrid regex-plus-Gemini intent router. For a student-level implementation, Node.js with Baileys for the gateway, Gemini for understanding and PowerShell/WMI for execution is a practical and explainable architecture.

Future versions can add more command modules, confirmation prompts for destructive actions, activity logs, additional authentication for sensitive commands and support for more languages. Thus, the proposed system can provide a useful foundation for personal productivity and small-scale IT administration. Its success depends on reliable access control, careful testing of intent recognition and responsible use of full-authority commands.

9. PROJECT INITIATION

9.1 Description of Problem Statement

Computing professionals and students who manage desktop workstations from a mobile environment face real limitations with existing tools: continuous video streaming that performs poorly on weak networks, recurring subscription costs, difficult phone-sized interfaces, privacy exposure through cloud-relayed sessions and no understanding of voice or natural-language requests. The problem is to design a low-cost operating-system automation engine, using an end-to-end encrypted messaging channel, that lets a user monitor, manage and automate a personal computer through simple text messages and voice notes.

9.2 Scope

- Conversational filesystem management: create, read, search, batch-rename and delete files and folders.
- Remote workstation diagnostics: memory, processor load, storage and uptime on demand.
- Screenshot delivery: single-frame desktop snapshots to check renders, downloads or windows.
- Emergency controls: lock the workstation and put the system to sleep remotely.
- Direct shell access: PowerShell (#ps) and Command Prompt (#cmd), restricted to the verified owner.
- **Out of scope:** continuous video streaming, access by anyone other than the whitelisted owner, and operating systems other than Windows.

9.3 Objectives

1. To remove the dependency on bandwidth-heavy remote-desktop software for day-to-day administrative tasks.
2. To use Google Gemini to transcribe voice notes and understand system commands in English, Hindi and regional languages.
3. To provide real-time hardware telemetry through native WMI, with a target response time under 2 seconds.
4. To deliver on-demand desktop screenshots to WhatsApp, with a target of under 3 seconds.
5. To automate filesystem organisation and allow restricted, owner-only terminal command execution.

10. STAKEHOLDERS AND RESOURCE IDENTIFICATION

10.1 Stakeholders

Stakeholder	Role / Interest	Expected Benefit
Owner / End User	Sends text, voice notes and commands from the phone	Remote, low-bandwidth control of own PC
Students / Small-scale IT administrators	Manage workstations and routine tasks	Free command bridge instead of paid tools
Platform and library providers (WhatsApp, Baileys, Google Gemini, Microsoft WMI)	Provide messaging channel, AI model and OS interfaces	Wider use of their platforms

10.2 Platform / Equipment / Resources

Hardware	Specification
Host workstation	Intel Core i3 / AMD Ryzen 3 (64-bit) or higher with 4 GB+ RAM
Mobile client	Any Android or iOS smartphone with WhatsApp installed
Display and network	GDI+ compatible graphics and an active broadband / Wi-Fi connection

Software	Requirement
Operating system	Microsoft Windows 10 / 11 (64-bit) or Windows Server 2019+
Runtime	Node.js v18.0.0+ and the NPM package manager
Core libraries	@whiskeysockets/baileys, docx, pdfkit, pino, dotenv
AI engine	Google Gemini 2.5 Flash / Pro multimodal API
Scripting shell	Windows PowerShell 5.1+ / 7.x with the WMI / CIM engine

10.3 Operating System

Microsoft Windows 10 / 11 (64-bit) or Windows Server 2019+. The final implementation should document the actual OS and software versions used.

11. REFERENCES

Publisher records, technical reports and documentation consulted for the selected papers and technologies:

[1] S. Aina, S. D. Okegbile, P. Makanju and A. I. Oluwaranti (2022), “An Architectural Framework for Facebook Messenger Chatbot Enabled Home Appliance Control System”, IGI Global.

[2] Gemini Team, Google (2023), “Gemini: A Family of Highly Capable Multimodal Models”, arXiv:2312.11805.

[3] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda and T. Scialom (2023), “Toolformer: Language Models Can Teach Themselves to Use Tools”, NeurIPS, arXiv:2302.04761.

[4] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz and M. Fritz (2023), “Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection”, arXiv:2302.12173.

[5] C. Zhang, L. Li, S. He et al. (2024), “UFO: A UI-Focused Agent for Windows OS Interaction”, Microsoft, arXiv:2402.07939.

[6] T. Xie, D. Zhang, J. Chen et al. (2024), “OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments”, NeurIPS, arXiv:2404.07972.

[7] WhiskeySockets Community, “Baileys: WhatsApp Web / Multi-Device API for TypeScript/JavaScript”, open-source project, https://github.com/WhiskeySockets/Baileys

[8] Microsoft Corporation, “Windows Management Instrumentation (WMI) Architecture and CIM Provider Model”, Microsoft Learn, https://learn.microsoft.com/en-us/windows/win32/wmisdk/